AI-Based Adaptive Precision Fertigation with Multi-Task Learning & Explainable Decision Support

> **An IoT, Cloud AI, and Computer Vision Driven System for Smart Plant Growth Monitoring, Irrigation & Fertilizer Management**

📌 1. Project Overview & Faculty Objective

Conventional agricultural irrigation and fertilization rely on static timers or manual inspection. This leads to water wastage, nutrient runoff, delayed disease diagnosis, and sub-optimal crop yield.

This project delivers a **Cloud-Connected Smart Fertigation System**. An IoT edge system (Raspberry Pi + Camera + Soil/Environmental Sensors) continuously captures plant growth imagery and environmental readings, uploading telemetry to the Cloud. The Cloud AI pipeline extracts **24 plant growth & visual parameters**, runs a **Multi-Task Neural Network (MTL)** paired with **Explainable AI (XAI)**, and presents real-time **Updates, Requirements, Decisions, and Suggested Actions** on an interactive Web Dashboard.

⚙️ 2. End-to-End System Working Mechanism

```
  [ FIELD / EDGE LAYER ]              [ CLOUD BACKEND LAYER ]              [ USER APPLICATION LAYER ]
 ┌──────────────────────────┐        ┌──────────────────────────────┐
┌──────────────────────────┐
 │ • Raspberry Pi 5         │        │ • FastAPI Ingestion Engine   │        │ • Live Telemetry Updates │
 │ • Camera Module 3        │        │ • OpenCV CV Pipeline         │        │ • Water/Fertilizer Needs │
 │ • Soil Moisture Sensor   │ ─────► │ • Multi-Task AI Engine       │ ─────► │ • AI Diagnosis & XAI     │
 │ • Temp & Humidity        │ (HTTP) │ • SHAP Explainability        │ (REST) │ • Actionable Suggestions │
 │ • Solenoid Valve/Pump    │        │ • PostgreSQL Database        │        │ • Remote Override Controls│
 └──────────────────────────┘        └──────────────────────────────┘
└──────────────────────────┘
```

The 4-Stage Operational Loop:
1. **Data Ingestion (Edge $\rightarrow$ Cloud)**: The field node captures periodic plant photos and reads soil moisture, temperature, and humidity. It packages these into a payload and transmits them to the Cloud API.
2. **Computer Vision Feature Extraction**: The Cloud backend processes plant imagery through OpenCV to extract 24 visual parameters across morphological, color, texture, and temporal metrics.
3. **Multi-Task Machine Learning & Forecasting**:
   - The 24 features + sensor data enter a shared multi-task neural network.
   - The model simultaneously predicts **Plant Growth Rate**, **Water Needs**, **Fertilizer Dose**, **Disease Risk**, and **Health Score**.

   - An LSTM / Time-Series model forecasts 24–48 hour future growth trends and advance water requirements.
4. **Explainable Decision Support (User App)**:
   - The web app renders live telemetry updates, explicit crop requirements, model decisions, and visual SHAP explanations showing *why* an action is recommended.
   - The system displays clear actionable suggestions (e.g., *"Irrigate 450ml water at 4:00 PM due to low soil moisture & rapid canopy expansion"*).

🔍 3. Visual & Sensor Parameters (Model Input)

The system extracts **24 core parameters** from plant imagery and environmental sensors:

Category	#	Parameter	Description / Extraction Technique
Morphological	1	Leaf length, width, height (Area)	Bounding box & contour area analysis
	2	Canopy Area	Total segmented green surface area (cm^2)
	3	Canopy Coverage Percentage	Ratio of green pixels to total foreground frame area
	4	Plant Height & Width	Vertical & horizontal extreme bounding points
	5	Leaf Count	Watershed segmentation & contour peak counting
	6	Number of Nodes	Skeletonization & branch junction detection
	7	Number of Flowers	HSV color segmentation (yellow/white/red hue clustering)
	8	Number of Fruits	Object detection / roundness shape filtering
	9	Stem Thickness	Caliper measuring across primary stem contour
	10	Projected Plant Area	Top-down convex hull projection
Color & Health	11	Leaf Color Index	Mean L*a*b* color space lightness & chromaticity
	12	Flower Color	Dominant HSV hue of floral regions
	13	Color Feature References	Color variance, standard deviation across channels
	14	RGB Histogram	256-bin distribution for Red, Green, and Blue channels
	15	HSV Values	Hue, Saturation, and Value mean & median values
	16	**Excess Green Index (ExG)**	Vegetation index formula: $ExG = 2G - R - B$
	17	Brown Regions	Chlorosis/necrosis pixel isolation via HSV threshold
	18	Visible Yellowing	Yellowing index derived from Hue shift towards yellow
Texture & Disease	19	Shape Features	Circularity, eccentricity, aspect ratio, solidity
	20	Texture Features (GLCM)	Contrast, dissimilarity, homogeneity, energy
	21	Health Indicators	Composite visual score derived from ExG & leaf area
	22	Disease Spots	Spot detection via adaptive thresholding & blob analysis
Temporal	23	Growth Rate	$\Delta \text{Canopy Area} / \Delta \text{Time}$ (daily growth rate)

| | 24 | Time-Series Parameters | Rolling 7-day mean of growth, moisture, and temperature |

🧠 4. Multi-Task AI & Explainability Architecture

A. Multi-Task Learning (MTL) Model
Rather than running 5 separate ML models, a single Multi-Task Neural Network uses a **shared encoder network** to extract common features, feeding into **5 specialized prediction heads**:
- **Head 1 (Growth Prediction)**: Continuous growth output (cm^2/day).
- **Head 2 (Water Requirement)**: Irrigation volume (ml or Liters/day).
- **Head 3 (Fertilizer Requirement)**: N-P-K nutrient dosage recommendation.
- **Head 4 (Disease Risk)**: Classification (Healthy / Early Blight / Chlorosis / Leaf Spot / Nutrient Deficiency).
- **Head 5 (Health Score)**: Continuous scale from 0 to 100.

B. Core Mathematical Formulas
1. **Excess Green Index (ExG)**:
   $$ExG = 2 \times \text{Green} - \text{Red} - \text{Blue}$$
2. **Growth Efficiency Index (GE)**:
   $$GE = \frac{\Delta \text{Plant Growth (Canopy Area)}}{\text{Total Water Supplied (Liters)}}$$
   Quantifies how efficiently the plant converts irrigation into structural biomass.

C. Explainable AI (XAI)
Every recommendation produced by the model is processed through **SHAP (SHapley Additive exPlanations)**. The web app presents:
- **Feature Contribution Breakdown**: Bar graphs showing which exact parameters drove a recommendation.
- **Visual Evidence**: Heatmap overlays showing diseased or stressed leaf areas causing alerts.

📋 5. Hardware & Software Requirements

Hardware Requirements
Component	Specification	Function
Microcontroller / Edge Computer	Raspberry Pi 5 (4GB or 8GB RAM)	Runs edge capture daemon & local sensor logging
Camera Module	Raspberry Pi Camera Module 3 (12MP)	Captures high-res plant imagery
Power Supply	Official Raspberry Pi 27W USB-C Power Adapter	Reliable field power supply
Storage & Accessories	32GB MicroSD Card, Tripod, Enclosure	Operating system, local buffer storage, weatherproof mounting
Sensors	Capacitive Soil Moisture Sensor v1.2, DHT22 / BME280	Measures soil water content, ambient temperature & humidity
Actuators (Optional Field Kit)	12V Solenoid Valve / 5V Relay Module	Automated irrigation execution

Software Requirements & Tech Stack

- **Languages**: Python 3.10+ (Backend/AI), JavaScript / TypeScript (Frontend Dashboard).
- **Computer Vision**: OpenCV (`opencv-python`), NumPy, SciPy, Pillow, scikit-image.
- **Machine Learning & XAI**: PyTorch / Scikit-Learn, LightGBM, SHAP, Pandas.
- **Backend Framework**: FastAPI, Uvicorn, Pydantic.
- **Database**: SQLite (Development) / PostgreSQL (Production) + SQLAlchemy.
- **Frontend Dashboard**: React (Vite), Vanilla CSS, Lucide Icons, Chart.js / Recharts.
- **Protocols**: REST API (HTTP JSON payloads) & WebSockets / MQTT for real-time telemetry.

📁 6. Target Repository Structure

```
fertigation-ai-system/
├── README.md                          # Complete system documentation (this file)
├── hardware/                          # Edge / Raspberry Pi scripts
│   ├── camera_capture.py              # picamera2 automated capture script
│   └── sensor_reader.py               # Soil moisture & DHT22 reading daemon
├── backend/                           # Cloud Backend Application
│   ├── main.py                        # FastAPI application entrypoint
│   ├── config.py                      # Environment & database configuration
│   ├── database/                      # SQLAlchemy models & migrations
│   │   ├── models.py                  # SensorData, ImageLog, PredictionLog schemas
│   │   └── connection.py
│   ├── cv_engine/                     # OpenCV 24-Parameter Extractor
│   │   ├── feature_extractor.py       # Main 24 parameter pipeline
│   │   ├── segmentation.py            # Canopy HSV & thresholding
│   │   ├── morphological.py           # Area, height, width, leaf count
│   │   └── color_texture.py           # ExG, HSV, brown/yellow indices, GLCM
│   ├── ml_engine/                     # AI & Machine Learning Pipeline
│   │   ├── mtl_model.py               # Multi-Task Learning PyTorch Architecture
│   │   ├── predictor.py               # Model inference runner
│   │   ├── xai_explain.py             # SHAP explanation generator
│   │   └── trained_models/            # Saved model weights (.pth / .pkl)
│   └── routers/                       # API Routes
│       ├── telemetry.py               # Ingest sensor & image payloads
│       ├── dashboard.py               # Serve frontend data (Updates, Requirements, Actions)
│       └── fertigation.py             # Irrigation & fertigation control endpoints
└── frontend/                          # Cloud Dashboard Web Application
    ├── package.json
    ├── vite.config.js
    ├── index.html
    └── src/
        ├── App.jsx
        ├── components/
        │   ├── LiveUpdates.jsx        # Real-time telemetry cards & camera feed
```

```
        │   ├── CropRequirements.jsx # Water & fertilizer requirements
        │   ├── AIDecisionsXAI.jsx # Multi-task predictions & SHAP charts
        │   ├── SuggestedActions.jsx # Action cards (Irrigate, Fertigate,
Alert)
        │   └── GrowthAnalytics.jsx  # Growth Efficiency Index &
historical charts
        └── styles/
            └── main.css           # Premium theme styling
```

🔑 7. Things You MUST Know Before Building

Before writing code, understand these critical concepts:

1. **How 24 Features Are Extracted Without Human Manual Measurement**:
   - You don't measure leaf length with a physical ruler. OpenCV uses a calibrated background grid or reference marker in the frame, calculating pixels-per-centimeter to derive real-world physical metrics automatically.

2. **How Excess Green (ExG) Works**:
   - Soil is brown/red, rocks are gray/white, and healthy plant leaves reflect high green light. $ExG = 2G - R - B$ zeroes out background noise and highlights crop leaves clearly even under varying outdoor sunlight.

3. **How Multi-Task Learning Benefits Precision Farming**:
   - Plant growth, water needs, fertilizer needs, and health score are biologically linked. Training a single network with a shared representation layer improves accuracy for all 5 tasks while drastically reducing model inference latency on cloud servers.

4. **Why Faculty Wants "Suggested Actions" Separate from "Decisions"**:
   - A **Decision** is raw AI diagnosis (e.g., *"Nitrogen Deficiency Detected, Water Stress 42%"*).
   - A **Suggested Action** is human-interpretable advice for the farmer (e.g., *"Action: Apply 15g Urea NPK fertilizer and increase evening watering cycle by 200ml"*).

5. **Cloud Simulation Capability**:
   - You do NOT need the physical Raspberry Pi connected immediately to build and test this project! We can build a **Telemetry Synthesizer & Image Simulator** in Python that pushes realistic plant growth data and images to your FastAPI backend, allowing complete end-to-end testing of the Cloud AI engine and React Dashboard before field deployment.

🚀 8. Phased Development Roadmap

We will build this project systematically in **6 phases**:

- [ ] **Phase 1: Project Skeleton & Database Schema** — Set up directory structure, FastAPI backend, and PostgreSQL/SQLite database models.
- [ ] **Phase 2: OpenCV 24-Parameter Extraction Engine** — Build and test the computer vision pipeline on sample plant images.

- [ ] **Phase 3: Multi-Task AI Model & SHAP XAI Engine** — Implement PyTorch MTL architecture, synthetic/training pipeline, and SHAP feature attribution generator.
- [ ] **Phase 4: Cloud API & Decision Engine** — Connect CV + MTL + XAI to FastAPI endpoints to compute Updates, Requirements, Decisions, and Suggested Actions.
- [ ] **Phase 5: Interactive Web Dashboard (React)** — Build the modern UI displaying telemetry updates, growth charts, SHAP plots, requirement cards, and action suggestions.
- [ ] **Phase 6: Edge IoT Client & Field Integration** — Write Raspberry Pi capture scripts, simulate cloud communication, and perform full system verification.

This document serves as the master specification for the AI-Based Precision Fertigation System.